

Esercizi esame scritti

Lambda Calcolo

Esercizio 1 [4 pt]

Beta riduzione

Un esercizio sul lambda calcolo potrebbe chiedere anche di fare cose diverse, ad esempio *applicare la capture avoiding substitution*

Applicare la **beta riduzione** alla seguente espressione fino a raggiungere un'espressione non ulteriormente riducibile (cioè in forma normale) o ad accorgersi che la derivazione è finita

$$(\lambda g. \lambda y. g y x)(\lambda f. f y)(\lambda k. k)$$

L'esercizio sulla beta riduzione potrebbe anche indicare una **strategia** da seguire (*call-by-name* o *call-by-value*) per la scelta del **redex**

Nella soluzione, mostrare **tutti i passi** di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la beta riduzione (**redex**) ed evidenziando le eventuali **α -conversioni**

Beta riduzione

Passo 1: inserire le parentesi sottintese

- La portata (lo scope) del **costruttore lambda** si estende **il più a destra** possibile
 $\lambda x. \lambda y. x y$ è la stessa cosa di $\lambda x. (\lambda y. (x y))$
- L'**applicazione funzionale** associa **a sinistra**
 $e_1 e_2 e_3$ è la stessa cosa di $(e_1 e_2) e_3$

$((\lambda g. (\lambda y. (gy)x))(\lambda f. fy))(\lambda k. k)$

Beta riduzione

Passo 1: inserire le **parentesi** sottintese

- La portata (lo scope) del **costruttore lambda** si estende **il più a destra** possibile
 $\lambda x. \lambda y. x y$ è la stessa cosa di $\lambda x. (\lambda y. (x y))$
- L'**applicazione funzionale** associa **a sinistra**
 $e_1 e_2 e_3$ è la stessa cosa di $(e_1 e_2) e_3$

$$((\lambda g. (\lambda y. (gy) x)) (\lambda f. fy)) (\lambda k. k)$$

Passo 2: individuare i **redex** e sceglierne uno da sviluppare

$$((\lambda g. (\lambda y. (gy) x)) (\lambda f. fy)) (\lambda k. k)$$

Unico redex in questo caso

Se il testo indica una **strategia** da seguire (**call-by-name** o **call-by-value**) la scelta del redex deve seguire tale strategia, **altrimenti** va bene un qualunque redex

Beta riduzione

Regola Beta riduzione

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

Passo 3: Controllare prima se occorre α -convertire

$$\underline{((\lambda g. (\lambda y. (gy) x)) (\lambda f. fy)) (\lambda k. k)}$$

Si può beta ridurre e arrivare in $(\lambda y. (\overset{e_1}{gy}) x) \{g = \overset{x:=e}{\lambda f. fy}\}$?

Solo se la sostituzione è capture avoiding: dato che $y \in FV(\lambda f. fy)$ dobbiamo prima α -convertire

$$((\lambda g. (\lambda y. (gy) x)) (\lambda f. fy)) (\lambda k. k) \equiv_{\alpha} ((\lambda g. (\lambda k. (gk) x)) (\lambda f. fy)) (\lambda k. k)$$

- $(\lambda y. e_1) \{x := e\} \equiv \lambda y. (e_1 \{x := e\})$ se $y \neq x$ e $y \notin FV(e)$
- $(\lambda y. e_1) \{x := e\} \equiv \lambda z. ((e_1 \{y := z\}) \{x := e\})$ se $y \neq x$ e $y \in FV(e)$
e z fresca
- $(\lambda x. e_1) \{x := e\} \equiv \lambda x. e_1$

$$FV(x) = \{x\}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$$

$$FV(\lambda x. e) = FV(e) \setminus \{x\}$$

Beta riduzione

Regola Beta riduzione

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

Passo 4: Adesso è possibile fare un passo di beta riduzione

$$\underline{((\lambda g. (\lambda k. (gk)x)) (\lambda f. fy)) (\lambda k. k)} \rightarrow (\lambda k. ((\lambda f. fy)k)x) (\lambda k. k)$$

Beta riduzione

Regola Beta riduzione

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

Passo 4: Adesso è possibile fare un passo di beta riduzione...

$$\underline{((\lambda g. (\lambda k. (g k) x)) (\lambda f. f y)) (\lambda k. k)} \rightarrow (\lambda k. ((\lambda f. f y) k) x) (\lambda k. k)$$

Passo 5: ... e continuare

- $(\lambda y. e_1) \{x := e\} \equiv \lambda y. (e_1 \{x := e\})$ se $y \neq x$ e $y \notin FV(e)$
- $(\lambda y. e_1) \{x := e\} \equiv \lambda z. ((e_1 \{y := z\}) \{x := e\})$ se $y \neq x$ e $y \in FV(e)$ e z fresca
- $(\lambda x. e_1) \{x := e\} \equiv \lambda x. e_1$

$$\underline{(\lambda k. ((\lambda f. f y) k) x) (\lambda k. k)} \rightarrow ((\lambda f. f y) \underset{e_1}{k}) x \{ \underset{x:=e}{k} := (\lambda k. k) \} = ((\lambda f. f y) (\lambda k. k)) x$$

N.B. qui non serve α -convertire: $f \notin FV(\lambda k. k) = \emptyset$

Beta riduzione

Regola Beta riduzione

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

Passo 6: ... e continuare

$$\underline{((\lambda f. fy)(\lambda k. k))}x \rightarrow ((\lambda k. k)y)x$$

Passo 7: ... e continuare

$$\underline{((\lambda k. k)y)}x \rightarrow yx$$

Beta riduzione

$$\underline{((\lambda g. (\lambda k. (gk)x)) (\lambda f. fy)) (\lambda k. k)} \rightarrow \underline{(\lambda k. ((\lambda f. fy)k)x)) (\lambda k. k)} \rightarrow$$

$$\underline{((\lambda f. fy) (\lambda k. k))}x \rightarrow \underline{((\lambda k. k)y)}x \rightarrow yx$$

Type Inference in OCaml

Esercizio 2 [4 pt]

Type inference

Indicare il **tipo** della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f x y z =  
  if x=[] then [y]  
  else match z with  
  | [] -> (y+1)::[]  
  | z1::z' -> if z1 then [y] else [];;
```

Type inference

Passo 1: ricostruire la struttura complessiva del tipo. In questo caso si tratta di una funzione **f** con tre parametri **x y z**

Dato che in OCaml le funzioni sono *curried*, i valori vengono presi uno alla volta, quindi la struttura complessiva del tipo è

$X \rightarrow Y \rightarrow Z \rightarrow \text{RIS}$

```
let f x y z =  
  if x=[] then [y]  
  else match z with  
  | [] -> (y+1)::[]  
  | z1::z' -> if z1 then [y] else [];;
```

Non è detto che il testo proponga sempre una funzione definita tramite **let**
L'esercizio potrebbe chiedere di inferire il tipo di una funzione anonima, un'applicazione di funzione, una lista, una tupla o **qualunque altra espressione** OCaml

Type inference

Passo 2: partire dalla struttura complessiva del tipo

$$\mathbf{X} \rightarrow \mathbf{Y} \rightarrow \mathbf{Z} \rightarrow \mathbf{RIS}$$

e provare a inferire i singoli tipi* (**X,Y,Z** e **RIS**), guardando il codice OCaml. Più precisamente, isolandone i **vincoli di tipo**

```
let f x y z =  
  if x=[] then [y]  
  else match z with  
  | [] -> (y+1)::[]  
  | z1::z' -> if z1 then [y] else [];;
```

- **X** = A list (da guardia primo if)
- **RIS** = Y list (da ramo then primo if)
- **Z** = B list (da pattern matching)
- **Y** = int (da primo caso p.m.)
- **Z** = bool list (da guardia secondo if)

*Convenzione: X,Y,Z come variabili di tipo per i parametri x,y,z, RIS come variabile di tipo del risultato, e A,B,C,... come variabili di tipo "fresche" per la definizione dei vincoli

Type inference

Passo 2: partire dalla struttura complessiva del tipo **$X \rightarrow Y \rightarrow Z \rightarrow RIS$** *Convenzione: X,Y,Z come variabili di tipo per i parametri x,y,z, RIS come variabile di tipo del risultato, e A,B,C,... come variabili di tipo "fresche" per la definizione dei vincoli

e provare a inferire i singoli tipi* (**X,Y,Z** e **RIS**), guardando il codice OCaml. Più precisamente, isolandone i **vincoli di tipo**

```
let f x y z =  
  if x=[] then [y]  
  else match z with  
  | [] -> (y+1)::[]  
  | z1::z' -> if z1 then [y] else [];;
```

- **X** = A list (da guardia primo **if**)
- **RIS** = Y list (da ramo then primo **if**)
- **Z** = B list (da pattern matching)
- **Y** = int (da primo caso p.m.)
- **Z** = bool list (da guardia secondo **if**)

Passo 3: risolvere i vincoli e arrivare al **tipo inferito**

```
X = A list  
Y = int  
Z = bool list  
RIS = int list
```

```
'a list -> int -> bool list -> int list
```

Funzioni (ricorsive) in OCaml

Esercizio 3 [7 pt]

Funzioni (ricorsive) in OCaml

Dato il seguente tipo di dato che descrive valute in Euro o Dollari:

```
type valuta =  
  | Euro of float  
  | Dollari of float;;
```

Definire in OCaml le **funzioni** `inEuro`, `somma_valute` e `separa_valute` con i seguenti tipi:

- `inEuro : float -> valuta -> valuta`
- `somma_valute : float -> valuta list -> valuta`
- `separa_valute : valuta list -> valuta list * valuta list`

Funzione **inEuro**

Passo 1: guardare il tipo e leggere la specifica

inEuro : float -> valuta -> valuta

inEuro

1. prende un numero reale t che rappresenta il tasso di cambio Euro/Dollaro
2. prende x di tipo valuta e restituisce y di tipo valuta
 - Se x è una valuta espressa in Dollari, la converte in Euro moltiplicandola per t
 - Se invece è già in Euro, la restituisce così com'è

Passo 2: in questo caso provare a tradurre la specifica:

```
let inEuro tasso x =
```

Funzione **inEuro**

Passo 1: guardare il tipo e leggere la specifica

inEuro : float -> valuta -> valuta

inEuro

1. prende un numero reale *t* che rappresenta il tasso di cambio Euro/Dollaro
2. prende *x* di tipo *valuta* e restituisce *y* di tipo *valuta*
 - Se *x* è una valuta espressa in Dollari, la converte in Euro moltiplicandola per *t*
 - Se invece è già in Euro, la restituisce così com'è

Passo 2: in questo caso provare a tradurre la specifica:

```
let inEuro tasso x =  
  match x with  
  | Euro e -> Euro e  
  | Dollari d -> Euro (tasso *. d);;
```

Funzione **inEuro**

Passo 1: guardare il tipo e leggere la specifica

inEuro : float -> valuta -> valuta
inEuro

1. prende un numero reale t che rappresenta il tasso di cambio Euro/Dollaro
2. prende x di tipo valuta e restituisce y di tipo valuta
 - Se x è una valuta espressa in Dollari, la converte in Euro moltiplicandola per t
 - Se invece è già in Euro, la restituisce così com'è

Passo 2: in questo caso provare a tradurre la specifica:

```
let inEuro tasso x =  
  match x with  
  | Euro e -> Euro e  
  | Dollari d -> Euro (tasso *. d) ; ;
```

Funzione `somma_valute`

Passo 1: guardare il tipo e leggere la specifica

```
somma_valute : float -> valuta list -> valuta  
somma_valute
```

1. prende un tasso di cambio `t` (come sopra) e una lista `lis` di elementi di tipo `valuta`
2. somma tutti gli elementi di `lis` restituendo un valore totale espresso in Euro. Eventuali valori in `lis` espressi in Dollari devono essere convertiti in Euro usando il tasso `t`

Passo 2: in questo caso scrivere la prima riga della funzione

```
let somma_valute tasso lis =
```

Funzione `somma_valute`

```
type valuta =  
  | Euro of float  
  | Dollari of float;;
```

`somma_valute : float -> valuta list -> valuta`

`somma_valute`

1. prende un tasso di cambio `t` (come sopra) e una lista `lis` di elementi di tipo `valuta`
2. somma tutti gli elementi di `lis` restituendo un valore totale espresso in Euro. Eventuali valori in `lis` espressi in Dollari devono essere convertiti in Euro usando il tasso `t`

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base**, quindi **(ii) pensare che “magicamente” la chiamata ricorsiva restituisce già i risultati nella forma richiesta, che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

```
let rec somma_valute tasso lis =  
  match lis with  
  | [] -> 0.0 (* Caso base: la somma di una lista vuota è 0 *)
```

Funzione `somma_valute`

```
type valuta =  
| Euro of float  
| Dollari of float;;
```

`somma_valute : float -> valuta list -> valuta`

`somma_valute`

1. prende un tasso di cambio `t` (come sopra) e una lista `lis` di elementi di tipo `valuta`
2. somma tutti gli elementi di `lis` restituendo un valore totale espresso in Euro. Eventuali valori in `lis` espressi in Dollari devono essere convertiti in Euro usando il tasso `t`

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base, quindi (ii) pensare che “magicamente” la chiamata ricorsiva restituisce già i risultati nella forma richiesta, che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

Caso: `x :: lis'`

<code>somma_valute tasso lis'</code>	<code>(* chiamata ricorsiva *)</code>
<code>+</code>	<code>(* combinazione *)</code>
<code>x</code>	<code>(* contributo elemento corrente *)</code>

Questo lo schema che va adattato rispetto al tipo degli elementi

Funzione `somma_valute`

```
type valuta =  
| Euro of float  
| Dollari of float;;
```

`somma_valute : float -> valuta list -> valuta`

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base, quindi (ii) pensare che “magicamente” la chiamata ricorsiva restituisce già i risultati nella forma richiesta, che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

```
let rec somma_valute tasso lis =  
  match lis with  
  | [] -> 0.0    (* Caso base: la somma di una lista vuota è 0 *)  
  | Euro x :: lis' -> somma_valute tasso lis' +. x  
    (* Somma in Euro *)  
  | Dollari x :: lis' -> somma_valute tasso lis' +. (x *. tasso)  
    (* Converti il Dollaro in Euro e somma *)  
;;
```


Funzione `somma_valute`

```
type valuta =  
  | Euro of float  
  | Dollari of float;;
```

`somma_valute : float -> valuta list -> valuta`

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base, quindi (ii) pensare che “magicamente” la chiamata ricorsiva restituisce già i risultati nella forma richiesta, che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

```
let rec somma_valute tasso lis =  
  match lis with  
  | [] -> 0.0    (* Caso base: la somma di una lista vuota è 0 *)  
  | Euro x :: lis' -> somma_valute tasso lis' +. x  
    (* Somma in Euro *)  
  | Dollari x :: lis' -> somma_valute tasso lis' +. (x *. tasso)  
    (* Converti il Dollaro in Euro e somma *)  
;;
```

Funzione `somma_valute`: alternative

Una funzione è **tail-recursive** se chiama se stessa in modo ricorsivo, ma **non** esegue alcun calcolo **dopo il ritorno della chiamata ricorsiva**

Passi per ottenere una versione Tail-recursive di una generica funzione `foo`

1. Trasformare la funzione ricorsiva `foo` in una **funzione ausiliaria** `foo-aux`.
Aggiungete un argomento in più: l'accumulatore `acc`
2. Scrivete una nuova versione "principale" della funzione, `foo_tr` che richiama la funzione ausiliaria e che passa il valore di ritorno del caso base originale come valore iniziale dell'accumulatore
3. Modificare la funzione **ausiliaria** per restituire `acc` nel caso base
4. Modificare il caso ricorsivo della funzione **ausiliaria** che deve fare del lavoro extra sull'argomento `acc`, prima della chiamata ricorsiva. Questo è l'unico passaggio che richiede un po' di ingegno

Funzione **somma_valute**: alternative

Versione Tail-recursive di **somma_valute**

```
let somma_valute tasso lis =  
  let rec somma_aux lst acc =  
    match lst with  
    | [] -> acc  
      (* Caso base: se la lista è vuota, restituisce l'accumulatore *)  
    | Euro x :: lis' -> somma_aux lis' (acc +. x)  
      (* Somma il valore in Euro *)  
    | Dollaro x :: rest -> somma_aux lis' (acc +. (x *. tasso))  
      (* Converti il Dollaro in Euro e somma *)  
  in  
    somma_aux lis 0.0  
;;
```

Funzione `somma_valute`: alternative

Versione che sfrutta le funzioni higher order

Possiamo usare `List.fold_left` per definire somma usando la ricorsione in modo implicito

Questa versione prevede i parametri (oltre alla lista `lis`):

- f : la **funzione** da applicare a ogni passo
- `0.`: il **valore iniziale** da cui partire con il calcolo (a sinistra rispetto a `lis`)

A ogni passo, richiama ricorsivamente f passandogli un valore via via aggiornato

```
let somma_valute tasso lis =  
  let f s v = match v with  
    | Euro e -> s +. e  
    | Dollari d -> s +. (tasso *. d)  
  in Euro (List.fold_left f 0. lis);;
```

Funzione `separa_valute`

Passo 1: guardare il tipo e leggere la specifica

`separa_valute : valuta list -> valuta list * valuta list`
`separa_valute`

1. prende una lista `lis` di elementi di tipo `valuta` e restituisce una coppia di liste
2. `(lis1, lis2)` dove `lis1` contiene gli elementi di `lis` espressi in Euro, e `lis2` quelli espressi in Dollari

Passo 2: scrivere la prima riga della funzione

`let separa_valute lis =`

Funzione `separa_valute`

Passo 1: guardare il tipo e leggere la specifica

`separa_valute : valuta list -> valuta list * valuta list`

`separa_valute`

1. prende una lista `lis` di elementi di tipo `valuta` e restituisce una coppia di liste
2. `(lis1, lis2)` dove `lis1` contiene gli elementi di `lis` espressi in Euro, e `lis2` quelli espressi in Dollari

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base**, quindi **(ii) pensare che “magicamente” le due chiamate ricorsive restituiscono già i risultati nella forma richiesta , che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

```
let rec separa_valute lis =  
  match lis with  
  | [] -> ([], [])  
  (* Caso base: una lista vuota restituisce due liste vuote *)
```

Funzione `separa_valute`

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base, quindi (ii) pensare che “magicamente” le due chiamate ricorsive restituiscono già i risultati nella forma richiesta , che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

- La chiamata ricorsiva restituisce una coppia di liste **(lis1,lis2)** dove **lis1** contiene gli elementi di **lis** espressi in Euro, e **lis2** quelli espressi in Dollari
- **Caso: Euro $x :: lis' : x$** verrà aggiunto, concatenandolo alla prima lista (**lis1**)
- **Caso: Dollaro $x :: lis' : x$** verrà aggiunto, concatenandolo alla secondo lista (**lis2**)

Funzione `separa_valute`

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base, quindi (ii) pensare che “magicamente” le due chiamate ricorsive restituiscono già i risultati nella forma richiesta , che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

```
let rec separa_valute lis =  
  match lis with  
  | [] -> ([], []) (* Caso base: una lista vuota restituisce due liste vuote *)  
  | Euro x :: lis' ->  
    let (lis1, lis2) = separa_valute lis' in  
    (Euro x :: lis1, lis2) (* Aggiungi l'elemento agli Euro *)  
  | Dollaro x :: rest ->  
    let (lis1, lis2) = separa_valute rest in  
    (lis1, Dollaro x :: lis2) (* Aggiungi l'elemento ai Dollari *)  
;;
```


Funzione `separa_valute`

Passo 3: individuare la soluzione del problema in **termini ricorsivi**, applicando il mantra: **(i) scrivere il caso base, quindi (ii) pensare che “magicamente” le due chiamate ricorsive restituiscono già i risultati nella forma richiesta , che vanno solo ricombinati con il contributo dell’elemento corrente della lista**

```
let rec separa_valute lis =  
  match lis with  
  | [] -> ([], []) (* Caso base: una lista vuota restituisce due liste vuote *)  
  | Euro x :: lis' ->  
    let (lis1, lis2) = separa_valute lis' in  
    (Euro x :: lis1, lis2) (* Aggiungi l'elemento agli Euro *)  
  | Dollaro x :: rest ->  
    let (lis1, lis2) = separa_valute rest in  
    (lis1, Dollaro x :: lis2) (* Aggiungi l'elemento ai Dollari *)  
;;
```

Funzione `separa_valute`: alternative

Versione che sfrutta le funzioni higher order

Possiamo usare `List.filter` per estrarre prima gli elementi in **Euro** e poi gli elementi **Dollari**. Questa funzione è infatti in grado di selezionare gli elementi della lista che soddisfano una determinata condizione, nel nostro caso il predicato `isEuro`

```
let separa_valute lis =  
  let isEuro x = match x with  
    | Euro _ -> true  
    | _ -> false  
  in let isDollari x = not (isEuro x)  
  in (List.filter isEuro lis, List.filter isDollari lis);;
```

Funzione `separa_valute`: alternative

Altra versione che sfrutta le funzioni higher order

Possiamo usare anche `List.fold_right` per estrarre prima gli elementi in **Euro** e in **Dollari**, scandendo la lista una sola volta

```
let separa_valute lis =  
  let f x (eur_lis,dol_lis) = match x with  
  | Euro _ -> (x::eur_lis,dol_lis)  
  | Dollaro _ -> (eur_lis,x::dol_lis)  
  in List.fold_right f lis ([],[])::
```

MiniCaml

Esercizio 4 [15 pt]

Estensioni di MiniCaml

Si estenda il linguaggio **MiniCaml** visto a lezione con un costrutto **LimFun** per definire "**funzioni su interi (non ricorsive) a dominio limitato**", cioè funzioni con un parametro di tipo intero, applicabili solo a valori in un intervallo definito al momento della definizione della funzione stessa

Applicare la funzione a valori al di fuori di tale intervallo genera un errore

Ad es. (in una sintassi concreta simile a OCaml):

```
LimFun x:{1..10} -> x+1
```

```
(* funzione di incremento, solo per valori tra 1 e 10 *)
```

Si mostri come deve essere modificato l'interprete **OCaml** del linguaggio

L'esercizio potrebbe chiedere invece di estendere MiniCaml con un nuovo tipo di dati e le funzioni per manipolarlo, oppure di aggiungere un nuovo costrutto

Estensioni di MiniCaml

Costrutto **LimFun** per definire "funzioni su interi (non ricorsive) a dominio limitato"

Fun of ide * exp (* Astrazione di funzione *)



Primo passo: è una astrazione funzionale nuova (perché quella già presente non basta), quindi serve estendere le espressioni, usando anche due ulteriori parametri che segnano gli estremi dell'intervallo di definizione. L'applicazione di funzione invece non cambia

```
type exp = ...  
| LimFun of ide * exp * exp * exp  
| Apply of exp * exp (* Applicazione di funzione *)
```

Estensioni di MiniCaml

Costrutto **LimFun** per definire "funzioni su interi (non ricorsive) a dominio limitato"

Closure of ide * exp * evT env

Secondo passo: analogamente a questa nuova astrazione corrisponde una nuova chiusura, perché quella già presente non basta

```
type exp = ...  
| LimFun of ide * exp * exp * exp  
| Apply of exp * exp (* Applicazione di funzione *)
```

```
type evT = ...  
| LimClosure of ide * int * int * exp * evT env
```

- il nome del parametro formale (ide)
- i due estremi dell'intervallo (int)
- il corpo della funzione dichiarata (exp)
- l'ambiente al momento della dichiarazione (evT env)

Estensioni di MiniCaml

Terzo passo: analogamente va estesa la funzione di valutazione sia per (i) la definizione (che richiede anche di valutare le espressioni che rappresentano gli estremi dell'intervallo) sia per (ii) l'applicazione (che richiede di valutare l'appartenenza all'intervallo)

```
Fun(i, a) -> Closure(i, a, s)
```

```
let rec eval (e: exp) (s: evT env) : evT =  
  match e with  
  | ...  
  | LimFun (arg, lim1, lim2, body) ->  
    (match eval lim1 s , eval lim2 s with  
     | Int l1, Int l2 -> LimClosure (arg, l1 , l2 , body, s)  
     | _ -> failwith "Error"  
    )
```


Estensioni di MiniCaml

Terzo passo: analogamente va estesa la funzione di valutazione sia per (i) la definizione (che richiede anche di valutare le espressioni che rappresentano gli estremi dell'intervallo) sia per (ii) l'applicazione (che richiede di valutare l'appartenenza all'intervallo)

```
| Apply(Den(f), eArg) ->  
  let fclosure = s f in  
  (match fclosure with  
  | Closure(arg, fbody, fDecEnv) ->  
    let aVal = eval eArg s in  
    let aenv = bind fDecEnv arg aVal in  
    eval fbody aenv  
  | _ -> failwith("non functional  
value"))
```

```
| Apply(eF, eArg) ->  
  let fclosure = eval eF s in  
  (match fclosure with  
  | LimClosure (arg, l1, l2, fbody, fDecEnv) ->  
    let aVal = eval eArg s in  
    (match aVal with  
    | Int n -> if n>=l1 && n<=l2 then  
      let aenv = bind fDecEnv arg aVal in  
      eval fbody aenv  
    else  
      failwith "Interval Error"  
    )  
  | _ -> failwith "Type Error"  
  )
```

Estensioni di MiniCaml

```
type exp = ...  
| LimFun of ide * exp * exp * exp  
| Apply of exp * exp (* Applicazione di funzione *)
```

```
type evT = ...  
| LimClosure of ide * int * int * exp * evT env
```

```
let rec eval (e: exp) (s: evT env) : evT =  
  match e with  
  | ...  
  | LimFun (arg, lim1, lim2, body) ->  
    (match eval lim1 s , eval lim2 s with  
      | Int l1, Int l2 ->  
        LimClosure (arg, l1 , l2 , body, s)  
      | _ -> failwith "Error"  
    )
```

```
| Apply(eF, eArg) ->  
  let fclosure = eval eF s in  
  (match fclosure with  
  ...  
  | LimClosure (arg, l1, l2, fbody,  
fDecEnv) ->  
    let aVal = eval eArg s in  
    (match aVal with  
    | Int n -> if n>=l1 && n<=l2 then  
      let aenv = bind fDecEnv arg aVal in  
      eval fbody aenv  
    else  
      failwith "Interval Error"  
    )  
    | _ -> failwith "Type Error"  
  )
```